

Relatório Técnico de Engenharia de Contexto: Mapeamento de Variáveis Internas, Gatilhos e Comandos de API no Manus.im (Free Tier)

1. Introdução e Fundamentos Arquitetônicos do Manus Event Stream

O Manus é reconhecido como um *action engine*, uma arquitetura de agente que transcende a mera resposta conversacional para executar tarefas complexas e automatizar fluxos de trabalho. Para usuários da versão gratuita, que oferece acesso ilimitado ao *Chat Mode* e a capacidade de fazer a transição para o *Agent Mode*, a compreensão dos mecanismos internos é fundamental para otimizar o consumo de recursos e garantir a continuidade das tarefas. A distinção entre o *Chat Mode* e o *Agent Mode* é operacional e financeira. O *Chat Mode* serve como um LLM conversacional primário para o *brainstorming* e a evolução de ideias. O *Agent Mode*, por sua vez, é acionado quando a intenção de execução se torna clara, iniciando o *Agent Loop* completo que concede ao Manus acesso ao ambiente *Sandbox* (incluindo *shell*, editor de texto e navegador). A versão Free do Manus impõe limites de crédito diário para o *Agent Mode*, tornando a gestão de cada execução crucial. O ponto de transição do *Chat Mode* para o *Agent Mode* representa o maior risco operacional e de custo para o usuário Free. Esta transição é sinalizada internamente pelo gatilho `runner_started` (detalhado na Seção 2.1), marcando o início do consumo de recursos caros. A arquitetura exige, portanto, que o usuário trate a solicitação de execução como um comando explícito, inserindo diretrizes que exijam uma pré-análise e um pedido de aprovação antes que o Manus acione internamente o processamento agentico.

1.1. A Coluna Vertebral da Reatividade: O Event Stream e a Rastreabilidade

O coração da reatividade e do autoaperfeiçoamento do Manus é o **Event Stream** (Fluxo de Eventos). Este fluxo atua como um registro imutável de tudo o que ocorre no sistema – desde a execução de código no *Sandbox* até a atualização do conhecimento no *Memory Bank*. As variáveis internas e os gatilhos listados neste relatório são os metadados anexados a este fluxo, garantindo que o agente possa auditar seu próprio progresso, aprender com os erros e retomar tarefas interrompidas.

A rastreabilidade proporcionada por este sistema se integra profundamente com o *Memory Bank* e o ambiente *Sandbox*. Cada execução é marcada por um `run_id`, que funciona como o ponto de ancoragem para correlacionar logs, artefatos, patches de conhecimento e feedback humano. Este rigor técnico é o que permite ao Manus funcionar como um sistema *agentico* em vez de apenas um modelo de linguagem, garantindo que o estado e o progresso sejam

persistentes e auditáveis.

2. Catálogo Exaustivo de Variáveis Internas e Gatilhos na Interface Chat

Os gatilhos e variáveis internos do Manus refletem seu estado, progresso e ciclo de autoaperfeiçoamento. O usuário não manipula diretamente a maioria desses gatilhos, mas o comportamento e as boas práticas de *prompting* podem influenciar quando e como eles são acionados ou referenciados no contexto de chat.

2.1. Gatilhos de Execução e Rastreabilidade (Sandbox e Logs)

Estes gatilhos monitoram o ciclo de vida da tarefa dentro do ambiente de execução do Manus (Sandbox), sendo essenciais para a auditoria de desempenho e a recuperação de falhas.

-  **run_id**: Identificador único para cada execução de uma tarefa. É essencial para correlacionar todos os artefatos gerados (logs, patches, PRs) e garantir a rastreabilidade, funcionando como o ponto de partida para qualquer análise de meta-aprendizado.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Para maximizar a utilidade de qualquer feedback (user_feedback_received), o usuário deve sempre incluir o run_id no prompt, especialmente ao reportar um erro ou submeter uma correção. Isso força o agente ou o sistema de suporte a correlacionar a entrada humana com a execução exata. Em prompts subsequentes, a menção explícita do run_id da tarefa anterior pode ser usada para reforçar a herança de contexto após uma falha ou interrupção.
-  **runner_started / runner_completed**: Sinaliza que o *runner* de execução iniciou ou finalizou com sucesso ou falha.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: O runner_started é o marcador do início do *Agent Mode* e, conseqüentemente, do consumo de créditos. A principal estratégia de gestão de custos é estruturar o prompt inicial exigindo uma pré-visualização do plano de ação. O usuário deve iniciar a interação com a condição: "Dê-me o plano detalhado no todo.md e peça aprovação (approval_granted) antes de acionar o runner_started." Isso garante que o Agent Mode seja ativado somente após a validação humana.
-  **test_passed**: Sinal de sucesso de um teste ou conjunto de testes.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Em tarefas que envolvem desenvolvimento de código, análise de dados ou relatórios baseados em fatos, o usuário deve solicitar ao Manus que conclua a tarefa com uma confirmação explícita de que a qualidade foi verificada. O comando "Gere o resultado e anexe um evento test_passed interno indicando a validação final" aumenta a confiança na saída antes que o workflow_completed seja acionado.
-  **test_failed**: Falha detectada em testes; deve gerar *issue/patch*.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Se o Manus reportar uma falha, solicitar imediatamente a análise do log gerado pelo evento test_failed. Em vez de simplesmente pedir que o agente tente novamente, o usuário deve fornecer um prompt de correção cirúrgico, baseado no diagnóstico de falha, economizando iterações potencialmente caras do *Agent Loop*.
-  **snapshot_saved**: Estado do ambiente salvo com o run_id, ficando pronto para

rollback.

- **Boas Práticas Operacionais no Chat (Free Tier):** Este gatilho é essencial para a resiliência. Em tarefas longas, que correm o risco de exceder o limite de tempo/token e "crashar", o usuário deve forçar o *checkpointing*. Instruir o Manus a explicitamente "Salvar o estado via `snapshot_saved` após a conclusão da fase X" melhora significativamente a performance de *herança de tarefas* se a sessão for abruptamente interrompida.
- 📦 **artifact_uploaded:** Um artefato (log, binário, relatório de cobertura) foi enviado para o *storage*.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Uma vez que a memória do chat pode se degradar, a melhor prática é externalizar o contexto de execução (Persistence-as-Context). Em vez de depender da memória conversacional, solicite que o Manus armazene dados críticos como `artifact_uploaded` e, posteriormente, refira-se ao nome do arquivo (por exemplo, "Use o `artifact_uploaded` nomeado `dados_finais.json` para o próximo passo").
- 🏁 **workflow_completed:** O ciclo completo de um *workflow* foi concluído (sucesso ou bloqueado), marcando o fim de um ciclo de trabalho.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Após a entrega do resultado final, o usuário deve pedir uma confirmação de que o `workflow_completed` foi acionado. Isso garante que o ciclo de feedback foi fechado e que a próxima interação pode ser tratada como um novo *seed prompt* e uma nova execução, evitando a confusão de contexto.
- 💓 **agent_heartbeat:** Sinal periódico de *liveness* (atividade) do Manus ou *runner*.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o Manus parecer travado ou demorar a responder, o usuário pode intervir solicitando um resumo do estado atual. Este comando implícito força o agente a verbalizar seu último *heartbeat* e a próxima etapa planejada no `todo.md`, ajudando a diagnosticar *loops* ou bloqueios.

O uso do Sandbox e o `artifact_uploaded` são mecanismos arquitetônicos que permitem ao Manus mitigar o problema inerente de memória do LLM ao salvar o estado de execução no disco (Persistence-as-Context). Para usuários que relatam problemas de continuidade, a melhor abordagem não é armazenar dados brutos no chat, mas sim usar o chat para *referenciar* artefatos e *solicitar snapshots*, transformando o `snapshot_saved` em um comando de *checkpoint* implícito.

2.2. 🧠 Gatilhos de Conhecimento e Autoaperfeiçoamento (Memory Bank)

Estes gatilhos governam como o Manus absorve, armazena e otimiza o conhecimento, influenciando diretamente a qualidade e a velocidade das respostas futuras.

- 📄 **prompt_seed_applied:** Um *seed prompt* foi injetado para iniciar uma execução, permitindo auditar a origem e a intenção inicial.
 - **Boas Práticas Operacionais no Chat (Free Tier):** O *seed prompt* inicial deve ser conciso e estável. Manter um prefixo de prompt estável é crucial para melhorar a taxa de acerto do KV-Cache, resultando em execuções mais rápidas e, consequentemente, menor consumo de crédito. O usuário deve aderir ao **Princípio da Clareza Única** para otimizar o uso da arquitetura subjacente.
- 📝 **prompt_template_updated:** Uma alteração foi feita nos templates de prompt

(promptTemplates.md), garantindo o uso das diretrizes mais recentes.

- **Boas Práticas Operacionais no Chat (Free Tier):** Se houver uma alteração súbita no comportamento de alto nível do Manus, é provável que um `prompt_template_updated` tenha ocorrido. O usuário deve responder injetando um *seed prompt* altamente específico para anular temporariamente a nova diretriz, restaurando o comportamento desejado.
-  **prompt_versioned:** Um template de prompt ou *seed* foi *versionado*, permitindo o controle de versão das instruções e comportamentos do Manus.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Ao relatar discrepâncias ou bugs, o usuário avançado deve sempre incluir a data e o contexto de execução, ajudando os desenvolvedores a correlacionar a falha com uma versão específica do template (`prompt_versioned`) para auditoria.
-  **ontology_updated:** Uma modificação foi feita na `ontology.md`, essencial para manter a coerência da arquitetura semântica do Manus.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o agente demonstrar confusão sobre a terminologia interna (por exemplo, a distinção entre um "patch" e um "artefato"), o usuário deve reorientar o prompt, usando a terminologia precisa do sistema alinhada com a ontologia para reafirmar a coerência semântica.
-  **embedding_update:** O *vector store* foi atualizado para um *chunk* ou *patch*, garantindo que a busca semântica esteja sempre atualizada.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o usuário fornecer um novo bloco significativo de conhecimento (um novo documento ou código-fonte), a prática mais eficiente é fechar a sessão, permitindo que o processamento do `embedding_update` ocorra em segundo plano. Só então a tarefa deve ser reaberta, garantindo que o novo conhecimento seja acessível via busca semântica.
-  **knowledge_patch_applied:** Um *patch* de conhecimento foi ativado ou usado em uma execução, registrando a frequência e eficácia de uso.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o Manus usar informações desatualizadas ou "alucinadas", o usuário pode solicitar um diagnóstico perguntando qual *knowledge patch* foi aplicado na execução anterior. Isso auxilia na refutação e correção da fonte da informação.
-  **self_compress_trigger / compressed_patch_suggested:** Acionado quando múltiplos *patches* similares se qualificam para compressão, impulsionando a otimização do Memory Bank.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Como o Manus otimiza ativamente sua memória, o usuário deve auxiliar proativamente o agente, fornecendo resumos concisos de conversas longas ("Task Summary Prompt"). Esta prática auxilia o `self_compress_trigger`, garantindo que o contexto essencial não se perca na entropia conversacional e melhora a qualidade da herança de tarefas.
-  **self_optimize_trigger:** Gatilho para rotinas de otimização (ajuste de parâmetros ou poda de *templates*), garantindo a eficiência contínua do Manus.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o comportamento do agente mudar sutilmente, o usuário deve considerar que o `self_optimize_trigger` pode ter sido acionado. Para isolar a causa da mudança (se é a otimização interna ou a instrução do usuário), é fundamental manter a estabilidade do *seed prompt*.
-  **user_feedback_received:** O *feedback* humano é recebido sobre um artefato ou execução, alimentando o processo de autoaperfeiçoamento.

- **Boas Práticas Operacionais no Chat (Free Tier):** O *feedback* humano é um mecanismo crítico de governança. Ele deve ser utilizado para mitigar o risco de *memory poisoning* (o armazenamento de informações falsas ou maliciosas). O *feedback* deve ser preciso, conciso e sempre incluir referência ao `run_id` e ao artefato relevante.
-  **documentation_sync_requested:** Solicitação para atualizar a documentação com base em mudanças de código ou *patch*.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Após a conclusão bem-sucedida de uma tarefa, o usuário pode instruir o Manus a formalizar o conhecimento, solicitando a geração de documentação. Este comando força o agente a fechar o ciclo de conhecimento, garantindo que o aprendizado da sessão seja registrado.

2.3. Gatilhos de Integração (Conectores e Governança Humana)

Embora muitos destes gatilhos estejam ligados a integrações de *software* (como GitHub), eles representam a capacidade do Manus de gerenciar tarefas estruturadas, planos de ação persistentes e a interação com a governança humana.

-  **todo_synced:** Sincronização entre `todo.md` e GitHub *issues*, mantendo o planejamento alinhado.
 - **Boas Práticas Operacionais no Chat (Free Tier):** O arquivo `todo.md` é a variável de estado mais importante para a manutenção do plano de longo prazo. O usuário deve tratá-lo como um comando explícito, solicitando: "Liste os itens do `todo.md`" ou "Adicione 'Revisar Secao X' ao `todo.md`". O uso ativo deste comando garante que o plano persista e mitiga a perda de contexto em sessões de herança.
-  **issue_created:** Permite que o Manus crie *issues* automaticamente no GitHub (ex: para reportar `test_failed` ou registrar novas funcionalidades).
 - **Boas Práticas Operacionais no Chat (Free Tier):** Após uma falha significativa (`test_failed` ou `loop_detected`), instrua o Manus a tratar a falha como uma *issue* e a resumir o diagnóstico. Este comando prepara o agente para a próxima tentativa, garantindo que a iteração de correção comece a partir de uma base de conhecimento estruturada.
-  **pr_created:** O Manus abre *Pull Requests* (PRs) no GitHub, anexando artefatos relevantes e *seed prompts*.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Mesmo sem a integração direta com o GitHub, o usuário pode solicitar que o Manus formalize o resultado da tarefa "como se fosse um PR". Isso força o agente a gerar um título, uma descrição e um resumo dos artefatos criados, conferindo uma estrutura formal ao resultado.
-  **pr_merged:** É acionado após a aprovação e mesclagem de um PR, atualizando o `projectBrief.md`.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Quando uma tarefa é concluída com sucesso, o usuário deve instruir o Manus a atualizar o `projectBrief.md` (ou um arquivo de resumo da sessão). Isso formaliza o conhecimento adquirido e garante que o Memory Bank o utilize como contexto de alto nível em tarefas subsequentes.
-  **pr_review_requested:** O Manus pode solicitar revisões de PRs.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Este gatilho pode ser usado como um mecanismo de *pausa para auditoria*. O usuário pode forçar o `pr_review_requested` no final de uma seção crítica, solicitando aprovação humana

("Revisar este trecho antes de prosseguir"), garantindo que o agente não gaste créditos avançando em uma direção incorreta.

- 📖 **changelog_updated**: O Manus pode gerar automaticamente entradas de *changelog* a partir de artefatos de execução.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Solicitar ao Manus que gere um *changelog* da última execução fornece um resumo estruturado e de fácil digestão das mudanças, essencial para documentação rápida.
- 🗃️ **decision_recorded**: Uma decisão explícita (ADR ou resposta de aprovação humana) é armazenada, garantindo o histórico claro das escolhas.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Sempre que uma ambiguidade for resolvida ou uma escolha crítica for feita, o usuário deve solicitar: "Registre esta decisão como *decision_recorded* para referência futura." Isso mitiga a recorrência do *ambiguity_detected* (Seção 2.4).
- ✅ / ❌ **approval_granted / approval_denied**: Registram as decisões humanas sobre ações críticas, garantindo um rastro de auditoria.
 - **Comando de Permissão (Crítico)**: É fundamental para a gestão de custos no Free Tier. O usuário deve exigir que o Manus solicite o *approval_granted* antes de iniciar o *Agent Mode* ou antes de tomar decisões irreversíveis que possam consumir grandes volumes de crédito.

2.4. 🚨 Gatilhos de Erro, Limitação e Governança (Resiliência)

Estes gatilhos governam a estabilidade, a segurança e, crucialmente para o Free Tier, a gestão de custos. Eles são indicadores diretos de falhas no *prompt engineering* ou de limites arquitetônicos.

- 🔁 **loop_detected**: O Manus identifica padrões de repetição que sugerem um *loop* infinito no Sandbox, interrompendo a execução e solicitando revisão humana.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: *Loops* consomem créditos rapidamente. Para preveni-los, injete restrições explícitas no *seed prompt* limitando o número de iterações do *Agent Loop*. Se o *loop* for detectado, o usuário deve intervir com um novo *seed prompt* focado em quebrar o ciclo, analisando a condição de repetição.
- ⚠️ **fallback_requested**: O Manus solicita intervenção humana ou *fallback* para um LLM de menor custo, gerenciando a complexidade e o custo.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Um *fallback* é um sinal de que a complexidade da tarefa está no limite do que o agente pode gerenciar eficientemente. O usuário deve aceitar o *fallback* e, em seguida, decompor a tarefa em partes menores e menos complexas para otimizar os recursos limitados do Free Tier.
- 🌐 **external_fetch_failed**: Sinaliza falha ao buscar recurso externo (como *paywall* ou rede).
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Se uma pesquisa de informação falhar, peça ao Manus para reportar o *external_fetch_failed* de forma transparente. Fornecer URLs de *backup* ou informações alternativas direcionadas ajuda o agente a contornar a falha de acesso.
- 🔍 **ambiguity_detected**: O sistema encontra múltiplas resoluções candidatas para uma ação ou símbolo, lidando com a incerteza.
 - **Boas Práticas Operacionais no Chat (Free Tier)**: Quando o Manus solicita

esclarecimento, o `ambiguity_detected` foi acionado. A melhor resposta é uma `decision_recorded` clara (Seção 2.3), eliminando a incerteza para a iteração atual e evitando que o agente fique preso em ciclos de deliberação.

-  **reference_link_broken:** Um link interno ou âncora não é mais válido, mantendo a integridade da base de conhecimento.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Este gatilho alerta para a degradação da base de conhecimento. Se o agente depender de uma fonte externa, o usuário deve verificar e, se necessário, fornecer um novo recurso estável.
-  **secret_detected:** Um *scanner* encontrou *secrets* (segredos) em *prompts* ou *commits*.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Este é um alerta de segurança crítico. A prática obrigatória é *nunca* incluir credenciais, chaves de API ou informações sensíveis em *seed prompts* ou artefatos de chat, mitigando o risco de exposição em logs não criptografados.
-  **cost_threshold_exceeded:** A execução ou os créditos diários cumulativos excederam o limite.
 - **Comando de Parada (Crítico):** Para o Free/Basic Tier, este gatilho é o mais importante. O usuário deve usá-lo como um comando de controle: "Se os custos atingirem 80% do limite, acione o `cost_threshold_exceeded` e me peça aprovação antes de prosseguir." Isso transforma o limite de custo em um ponto de auditoria programável, gerenciando o risco de um *Agent Loop* caro e descontrolado.
-  **maintenance_mode:** O sistema entra em modo de manutenção.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Lentidão ou falhas não explicadas podem indicar o `maintenance_mode`. O usuário deve planejar tarefas complexas para horários de menor tráfego ou aguardar a notificação de status.
-  **tool_scope_changed:** As ferramentas/modos disponíveis para o Manus foram atualizados.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Se o conjunto de ferramentas disponíveis for alterado, o usuário deve reavaliar e potencialmente refinar seu *seed prompt* para garantir que ele otimize o uso das novas ferramentas.
-  **privacy_consent_changed:** O usuário altera o consentimento ou as configurações de memória.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Alterar o consentimento deve ser acompanhado de uma instrução explícita no chat para "limpar a memória associada a esta thread", garantindo que o Memory Bank obedeça à alteração do consentimento.
-  **export_bundle_ready:** O pacote NDJSON/ND foi produzido para consumo por NotebookLM ou terceiros.
 - **Boas Práticas Operacionais no Chat (Free Tier):** Devido ao risco de perda de contexto em sessões interrompidas, a melhor prática de preservação do trabalho é solicitar o `export_bundle_ready` após a conclusão de uma tarefa, garantindo um backup estruturado do contexto e dos artefatos.

3. Comandos de API Implícitos e Modos de Interação no Chat (Free Tier)

No Manus, a interação em linguagem natural frequentemente atua como um comando de API

que manipula o estado interno do agente, acionando os gatilhos descritos. A gestão da transição de modo e a persistência do contexto são os comandos mais críticos que os usuários podem exercer.

3.1. O Comando de Transição: Ativação do Agent Mode (Runner Start)

A arquitetura do Manus é projetada para reconhecer e agir mediante a *intenção de execução*. Frases de comando como "Execute esta tarefa", "Construa o website X" ou "Inicie o Agent Mode agora" servem como o comando implícito que aciona o `runner_started`. No entanto, como o *Agent Mode* consome créditos, este comando deve ser mitigado.

A melhor prática de comando para esta transição é incluir uma cláusula de governança explícita no *seed prompt*. Por exemplo: "Inicie o Agent Mode, mas primeiro, detalhe o plano no `todo.md`, aguarde meu `approval_granted` e use o `run_id` para correlacionar o log de custo." Isso transforma o início do *runner* em um processo de duas etapas, protegendo contra o consumo inesperado de recursos.

3.2. Manipulação de Contexto Persistente (`todo.md` e Arquivos)

Uma limitação reconhecida dos agentes LLM, especialmente durante a herança de tarefas ou após *crashes*, é a dificuldade em manter o contexto total, muitas vezes lembrando apenas cerca de 50% da conversa anterior. O Manus contorna essa limitação usando arquivos no *Sandbox* para salvar resultados intermediários e notas, como o `todo.md`, em vez de depender exclusivamente do contexto do chat.

O `todo.md` atua como a variável de estado primária para o planejamento, persistindo o plano de execução mesmo quando o contexto conversacional é truncado. Portanto, o usuário pode "escrever no disco" do Manus por meio de comandos no chat, garantindo a continuidade do estado.

Comandos Implícitos de Manipulação de Estado:

- `/todo [nova tarefa]`: Comando explícito para adicionar itens ao plano (aciona o `todo_synced`).
- `@manus leia o arquivo X`: Força o carregamento de contexto de um `artifact_uploaded` anterior, garantindo que os dados de execução persistentes sejam injetados novamente no contexto de trabalho.
- Crie um Task Summary Prompt para o arquivo `'resumo.txt'`: Direciona o agente a formalizar o progresso no *Sandbox* (um pré-comando para `snapshot_saved`), que será mais estável do que o histórico do chat.

3.3. Comandos de Engenharia de Contexto para Estabilidade

A eficiência do *Agent Mode* está intrinsecamente ligada à forma como o contexto é processado, especificamente a otimização do KV-Cache.

- **Comandos de Estabilidade (KV-Cache)**: O princípio de que a estabilidade do *prompt prefix* é fundamental para o desempenho se traduz na prática de evitar a edição ou anulação de instruções anteriores.
- **Comando de *Append-Only***: O usuário deve tratar novas instruções como adendos ao plano existente, citando o ponto de partida do `run_id` atual, em vez de exigir uma reescrita completa da tarefa. Essa abordagem de *append-only* (apenas anexar) otimiza a reutilização do cache de contexto, o que, por sua vez, resulta em execuções mais

rápidas e econômicas.

O processo de feedback humano (`user_feedback_received`) é, metaforicamente, o comando de API mais direto para o processo de autoaperfeiçoamento do Manus. O *feedback* deve ser granular, focado em passos de execução ou artefatos específicos (com referência ao `run_id`), evitando o feedback vago que é difícil de correlacionar com eventos internos e que pode levar à degradação da memória.

A tabela a seguir resume as ações do usuário que atuam como comandos de API, mapeando-as para os gatilhos internos e o propósito operacional no *Free Tier*.

Tabela: Mapeamento de Comandos de Interação e o Ciclo de Vida do Agente

Ação do Usuário (Comando Implícito no Chat)	Gatilhos/Variáveis Chave Acionados	Finalidade Operacional no Free Tier	Impacto na Continuidade (Boas Práticas)
Iniciar Execução/Agente Mode	<code>runner_started</code> , <code>prompt_seed_applied</code>	Transição do modo Chat (gratuito) para o modo Agent (crédito)	Crítico para Gestão de Custos. Deve ser condicional (<code>approval_denied</code> é o fallback).
Fornecer um "Task Summary Prompt"	<code>snapshot_saved</code> , <code>self_compress_trigger</code> (indiretamente)	Preservar o contexto antes de um corte de sessão ou herança de tarefa.	Mitigação de Context Loss. Cria um ponto de retomada estável.
Solicitar atualização de <code>todo.md</code>	<code>todo_synced</code> , <code>decision_recorded</code>	Gerenciar o estado de planejamento e o foco do agente.	Mantém o plano de execução persistente, desacoplado da memória conversacional.
Enviar Feedback Qualitativo	<code>user_feedback_received</code> , <code>run_id</code> (meta-dado)	Alimentar o processo de autoaperfeiçoamento e Model Armor.	Aumenta a rastreadibilidade e previne <i>memory poisoning</i> .
Solicitar o estado do ambiente	<code>agent_heartbeat</code> (informação)	Diagnosticar o progresso ou a causa de um aparente travamento.	Quebra o <i>loop</i> (potencialmente evitando <code>loop_detected</code>) e força a verbalização do plano atual.

4. Estratégias de Preservação de Contexto e Otimização para o Free Tier

A análise dos gatilhos e comandos revela que o desafio central para o usuário avançado do Manus reside na gestão do estado (contexto) e na otimização da eficiência da execução para maximizar o uso limitado de créditos do *Free Tier*. As estratégias operacionais devem priorizar a resiliência arquitetônica.

4.1. Estratégias de Preservação de Contexto (Mitigando Falhas de

Herança)

A inconsistência na herança de tarefas, onde apenas uma fração do contexto é mantida, é um problema recorrente. A solução arquitetônica reside em externalizar o contexto operacional para o *Sandbox*, aproveitando os gatilhos de persistência.

- **Adoção do *Task Summary Prompt*:** A prática sugerida de criar um resumo conciso do progresso antes que a sessão atinja o limite é crucial. Este resumo deve ser formatado para ser injetado como um *seed prompt* estável na próxima iteração, servindo como o ponto de partida oficial, reforçando o *snapshot_saved*.
- **Referência Externa ao Contexto:** Em vez de digitar dados brutos, o usuário deve comandar o Manus a salvar/carregar contexto via *artifact_uploaded* e *snapshot_saved*. Essa estratégia de desacoplamento garante que a execução use dados persistentes, que são mais robustos do que o histórico de chat.
- **Controle sobre *todo.md*:** A utilização de comandos explícitos para manipular o *todo.md* (*todo_synced*) garante que o plano de execução persista, mesmo que o agente comece a próxima sessão com uma memória conversacional truncada.

4.2. Engenharia de Prompt para Estabilidade e Eficiência

No *Free Tier*, cada iteração desperdiçada devido a um *loop* ou a uma baixa taxa de acerto do KV-Cache se traduz em créditos perdidos. A engenharia de prompts deve, portanto, ser focada na estabilidade e na eficiência, tratando o prompt como um prefixo estável e *append-only*. A eficiência do KV-Cache, facilitada pela estabilidade do *prompt prefix* (*prompt_seed_applied*), resulta diretamente em execuções mais rápidas. O usuário deve evitar prompts que exijam que o agente reescreva ou modifique extensivamente o contexto anterior. Em vez disso, a instrução deve solicitar a geração de um novo artefato (um "patch") que complemente ou corrija o trabalho anterior (*knowledge_patch_applied*).

4.3. Gerenciamento Pró-ativo de Limitações e Erros

O usuário avançado deve tratar os gatilhos de erro não como falhas, mas como pontos de controle para intervenção.

- **Controle de Custo (Free Tier):** O gatilho *cost_threshold_exceeded* é a métrica primária de eficiência. A prática de exigir o *approval_granted* antes que o *runner_started* consuma recursos é a defesa mais forte contra o esgotamento prematuro dos créditos.
- **Reação a Falhas:** Quando *test_failed* ou *loop_detected* ocorrerem, a intervenção do usuário deve ser um novo *seed prompt* cirúrgico. Este prompt deve solicitar a análise do log ou do artefato de erro (forçando a transparência) em vez de simplesmente repetir a tarefa, garantindo que o agente aprenda e evite o erro na próxima iteração.

4.4. Conclusões: A Meta-Interação

A análise detalhada dos mecanismos internos do Manus.im demonstra que o controle eficiente do agente, especialmente na versão gratuita, depende da **meta-interação**. O usuário mais proficiente não fala *com* o agente, mas sim *para* o Event Stream e o Agent Loop. Ao utilizar a terminologia técnica no prompt (por exemplo, "Use o *run_id* X para carregar o *snapshot_saved* e continuar a tarefa, evitando o *loop_detected* anterior"), o usuário maximiza o

controle e a auditabilidade do processo. Esta abordagem transforma a interface de chat de uma simples conversa para uma linha de comando de engenharia de contexto, permitindo que o usuário utilize a arquitetura de persistência e autoaperfeiçoamento do Manus para compensar as limitações do *Free Tier* (restrição de créditos e risco de perda de contexto).

A estratégia arquitetônica de Manus de persistência de estado (via `artifact_uploaded` e `snapshot_saved`) e autoaperfeiçoamento (via `user_feedback_received` e `self_optimize_trigger`) exige que o usuário abandone o modelo conversacional e adote uma abordagem de engenharia de *workflow* para obter resultados consistentes e econômicos.

Tabela: Mapeamento de Arquitetura Interna para Fricção do Usuário

Gatilho Interno	Fricção Típica do Usuário (Sintoma)	Estratégia de Mitigação Direta
<code>cost_threshold_exceeded</code>	Créditos diários esgotados de forma inesperada.	Exigir <code>approval_granted</code> antes de <code>runner_started</code> .
<code>workflow_completed</code> / Nova Tarefa Herdada	Perda de 50% do contexto na transição de sessão.	Forçar <code>snapshot_saved</code> e <code>todo_synced</code> proativos (checkpointing).
<code>loop_detected</code>	Agente preso, uso excessivo de tokens/tempo.	Definir limites de iteração explícitos no <i>seed prompt</i> .
<code>ambiguity_detected</code>	Manus fazendo perguntas vagas ou pedindo escolhas óbvias.	Resposta com um <code>decision_recorded</code> conciso.
<code>prompt_seed_applied</code>	Desvio do objetivo inicial na execução.	Manter a estabilidade do prefixo do prompt (KV-Cache Optimization).

Referências citadas

1. Manus: Hands On AI, <https://manus.im/>
2. Introducing Manus Chat Mode - FREE & UNLIMITED to all - YouTube, <https://www.youtube.com/watch?v=gwHg7SrS81E>
3. Hands On AI - Manus, <https://manus.im/home>
4. How to Chat with Manus AI Without Executing Agents - YouTube, <https://www.youtube.com/watch?v=0s9W0OsT7V4>
5. Manus tools and prompts · GitHub, https://gist.github.com/jlia0/db0a9695b3ca7609c9b1a08dcbf872c9?permalink_comment_id=5478806
6. Feature Request: Warn users before hitting the task length limit for ..., https://www.reddit.com/r/ManusOfficial/comments/1k90zuu/feature_request_warn_users_before_hitting_the/
7. In-depth technical investigation into the Manus AI agent, focusing on its architecture, tool orchestration, and autonomous capabilities., <https://gist.github.com/renschni/4fbc70b31bad8dd57f3370239dccd58f>
8. Context Engineering for AI Agents: Lessons from Building Manus, <https://manus.im/hi/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
9. Turn 1 Prompt into a Fully Functional AI Agent! (Manus AI) - YouTube, <https://www.youtube.com/watch?v=kYzEBIzp7V8>
10. Vertex AI Agent Engine Memory Bank overview - Google Cloud Documentation, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/memory-bank/overview>
11. Agent mode / Chat mode : r/ManusOfficial - Reddit, https://www.reddit.com/r/ManusOfficial/comments/1llpd14/agent_mode_chat_mode/